Unit 2 Review

Everything here is from Unit 2. Find the skill you need, then write your own version for your app.

Teachable Machine

Train a model. A **class** is one label. Its photos are the examples. **Train Model** looks for patterns that tell the classes apart.

Test it on photos it has never seen. Before training, move a few photos of each class into a held-back folder. In **Preview**, switch **Input** on and choose **File**. A photo added to a class is training data, so it can no longer be a test.

Share it. Export Model, then **Upload my model**, then **Upload**. Copy the link.

Classifiers

Training data is a dictionary. Each key is a label. Each value is a list of example questions, worded differently.

```
from mlms import TextClassifier

plants = {
    "water": ["how often do I water it?",
              "is the soil too dry?",
              "can I water too much?"],
    "light": ["does it need sun?",
              "can it live in shade?",
              "where should the pot go?"]
}
bot = TextClassifier()
bot.train(plants)
```

Classify, then reply. `classify` hands back two values. Give every label its own reply.

```
q = "how much sun does it want?"
label, confidence = bot.classify(q)
if label == "water":
    print("Bot: Water when the soil is dry.")
elif label == "light":
    print("Bot: Bright light, not direct sun.")
```

Say "I don't know". `confidence` runs from 0 to 1. Below your cutoff, the bot admits it is guessing.

```
if confidence > 0.6:
    print("Bot: That is about", label)
else:
    print("Bot: Sorry, I don't know.")
```

A chat loop. `while True` repeats until `break` runs.

```
while True:
    question = input("You: ")
    if question.lower() == "quit":
        break
```

Recommenders

Items are a dictionary in a dictionary. Every item gets the same attributes, with numbers from 0 to 2.

```
games = {
    "chess":    {"fast": 0, "team": 0},
    "soccer":   {"fast": 2, "team": 2},
    "tag":      {"fast": 2, "team": 1},
    "checkers": {"fast": 1, "team": 0}
}
```

Rate, then recommend. `recommend` gives back `(name, score)` pairs, best first. A `for` with two names takes each pair apart.

```
from mlms import TextRecommender

rec = TextRecommender()
rec.set_items(games)
rating = int(input("Rate chess, 1 to 5: "))
rec.add_rating("jo", "chess", rating)
for name, score in rec.recommend("jo"):
    print(name, round(score, 2))
```

Different attributes, different picks. Same items and ratings, new attributes: the list changes. A recommender can only suggest what its attributes describe.

Word pairs: predict the next word

Count with `.get`. `.get(key, 0)` gives 0 when the key is not there yet.

```
counts = {}
for word in "to be or not to be".split():
    counts[word] = counts.get(word, 0) + 1
```

Count which word follows which. One dictionary of counts per word. The last word has nothing after it.

```
words = "the cat sat on the mat".split()
successors = {}
for i in range(len(words) - 1):
    word = words[i]
    next_word = words[i + 1]
    if word not in successors:
        successors[word] = {}
    count = successors[word].get(next_word, 0)
    successors[word][next_word] = count + 1
```

Generate text. Start at a word, pick a follower, repeat. `pick_next` (already in your starter) picks bigger counts more often.

```
result = ["the"]
for i in range(5):
    if result[-1] not in successors:
        break
    options = successors[result[-1]]
    result.append(pick_next(options))
print(" ".join(result))
```

Change the corpus, change the voice. Same code, different text file, different writing. Train one model per file in a loop.

Two-word keys and temperature. A slice and `join` make a two-word key. Each count to the power `1 / temperature`: small temperature, safe picks; big temperature, surprises.

```
key = " ".join(words[0:2])   # the cat
temperature = 0.5
weight = 3 ** (1 / temperature)   # 9.0
```

Language models

Ask once. The system prompt says who the model should be. Your message is what you ask. You get the reply back as a string.

```
from simple_llm import ask_once

role = "You are a friendly librarian."
question = "What is a good mystery book?"
reply = ask_once(role, question)
print(reply)
```

Prompt checklist: write a stronger system prompt. Task: what to do. **Role:** who the model is. **Audience:** who it is talking to. **Format, length, tone:** how the answer should look and sound. **Examples:** a sample of what you want. **Thinking step:** ask it to think first. Change one part at a time.

```
system_prompt = """You are a chess coach.
Suggest one move and explain it.
The player is 12 and new to chess.
Answer in two short, friendly sentences.
Example: Move your knight here.
It now guards two squares.
Look at the whole board before you answer."""
```

Remember the conversation. `chat` does not change `history`. After each reply, add your message, then the bot's.

```
from simple_llm import chat

role = "You are a friendly helper."
history = []
message = "My dog is called Pip."
reply = chat(role, history, message)
history.append({"role": "user",
                "content": message})
history.append({"role": "assistant",
                "content": reply})
```

Stay on topic. Say the topic in the prompt, and what to do when asked about anything else. Then test it with an off-topic question, and with a trick to break the rules.

Check its answers. A language model can be wrong, and sound sure. Check facts in a source you trust. Watch for exact numbers, dates, quotes, and things that do not exist.

Image prompts

Make an image. `generate_image` takes the system prompt (the look of every picture), then the request. It saves a file: refresh the file list and open it.

```
from simple_image_llm import generate_image

animal = input("Animal: ")
place = input("Place: ")
request = f"A {animal} resting in a {place}."
look = "You paint soft watercolors."
filename = generate_image(look, request)
print("Saved as", filename)
```

Testing your AI

Try odd input on purpose. An empty line, gibberish, a word where a number should be. `try` runs its lines; if `int()` cannot read the text, Python jumps to `except` instead of crashing.

```
answer = input("How many pets? ")
if answer.strip() == "":
    print("Type a number.")
else:
    try:
        pets = int(answer)
    except ValueError:
        print("That was not a number.")
```

Look for unfair answers. Pretend to be someone very different from you, and type what they might type. Does the training data, the items list or the prompt leave anyone out?